UNIFAAT 2026.2

01 - Servidor de Arquivos Estáticos e DOM

Prof. Luan Lourenço

Avisos da disciplina

- **Não é um curso de HTML e CSS** — é frontend como peça de um sistema maior
- Continuidade direta de **Desenvolvimento Web** (o backend do semestre passado)
- Vai ser mais **prático** que Desenvolvimento Web
- **1 prova teórica** e **1 prova prática**
- Carga horária reduzida por causa dos feriados

Combinados de sala

- **Meta-aprendizagem:** aprendendo a aprender
- Usem IA, mas com uma **base sólida** por baixo
- Professor disponível fora do horário de aula (chama no zap)
- Conversa paralela, por favor, fora da sala

Bora lá!

O repositório do bimestre

Todo o código das aulas do bimestre mora num único repositório no GitHub.

`github.com/luan-tavares/unifaat-frontend-bimestre-01`

É nele que o projeto compartilhado (backend + frontend) vai evoluindo aula a aula.

O que é Frontend?

```
# processamento no client
Tudo que gera a GUI — montar tela,
reagir a cliques, atualizar o que o
usuário vê — acontece no navegador,
não no servidor.
```

```
# não existe sozinho
Depende do backend dentro da
arquitetura client/server.
Só existe porque antes houve uma
requisição, uma resposta e um servidor.
```

O objetivo do curso: trabalhar com projetos reais, entendendo o frontend como peça de um sistema maior.

Conteúdo dinâmico vs. estático

```
# dinâmico
Gerado sob demanda, na requisição.
Não existe como arquivo pronto.

• processado pelo backend
• dados variam a cada requisição
• HTML, JSON, CSS gerados na hora
```

```
# estático
Já existe persistido no servidor,
esperando ser requisitado.

• imagens, fontes, vídeos
• HTML, CSS, JS servidos prontos
• apenas copiado para o client
```

Dinâmico se calcula a cada pedido. Estático só se entrega.

Por que não servir estático pelo backend?

```
# nginx — baixo overhead
requisição → sistema de arquivos
caminho direto e eficiente
sem intermediários
```

```
# backend — alto overhead
rota → middlewares → controller
→ leitura de disco
linguagem interpretada, garbage collector
e event loop no caminho
```

Passar pelo ciclo do Node só pra entregar um arquivo é overhead desnecessário: de **1 a 2 ordens de grandeza** de diferença em throughput.

O Nginx resolve a requisição direto contra o sistema de arquivos.

Nginx como proxy reverso

O Nginx decide com base no **método** e no **path** da requisição HTTP:

```
# arquivo estático
resolve direto —
sem passar pelo backend
```

```
# conteúdo dinâmico
repassa a chamada
pro backend processar
```

Um único ponto de entrada, dois destinos possíveis.

O bloco server no Nginx

default.conf

```
server {
  listen 80;
  root /var/www;
  index index.html;
  location / {
    autoindex on;
    try_files $uri $uri/ =404;
  }
  error_page 404 /404.html;
}
```

Match por especificidade

O Nginx escolhe o **prefixo mais longo** que casa com o path.

default.conf

```
location / {
  try_files $uri $uri/ =404;   # arquivo → diretório → 404
}
location /imagens/ {
  try_files $uri =404;         # só arquivo → 404 (mais específico)
}
```

Vale a especificidade, não a ordem de escrita.

HTML semântico

Só de ler o HTML dá pra entender o que cada parte representa — e navegador, buscador e leitor de tela também entendem.

so-divs.html

```
<div class="header">
  <div class="menu">…</div>
</div>
<div class="content">…</div>
<div class="footer">…</div>
```

semantico.html

```
<header>
  <nav>…</nav>
</header>
<main>
  <article>…</article>
</main>
<footer>…</footer>
```

`header`, `nav`, `main`, `section`, `article`, `aside`, `footer`: estrutura com significado.

O DOM

Document **O**bject **M**odel. HTML, CSS e JS trafegam pela rede como **strings** — texto puro.

O navegador precisa transformar esse texto em algo que dê pra **manipular**.

`Texto não se manipula. Objeto se manipula.`

De string para objeto

```
# 1 · JSON chega como string
JSON.parse() →
objeto manipulável
```

```
# 2 · HTML chega como string
o navegador faz o parsing →
gera o DOM
```

Fonte (string) → interpretação (parsing) → resultado em memória → manipulação via JS. O DOM vive na **memória do navegador** (o client).

A mesma lógica do JSON, aplicada ao HTML.

A estrutura em árvore

dom-tree

```
html            ← nó raiz
├── head
│   └── title
└── body
    ├── h1          ┐
    ├── p           ├ irmãos (mesmo pai: body)
    └── button      ┘
```

Cada tag HTML vira um `HTMLElement` — um nó da árvore, com **pai**, **filhos** e **irmãos**.

JS de frontend: Vendor API

```
# node.js (backend)
sem acesso ao DOM
sem document, sem window
APIs de sistema de arquivos e rede
```

```
# navegador (frontend)
API própria pra manipular o DOM
JS orientado a eventos
ponto de entrada: document
```

O objeto `document` é a porta de entrada pra toda manipulação do DOM via JavaScript.

Criando e inserindo elementos

console.js

```
// cria o elemento — ainda fora da árvore (retorna HTMLElement)
const el = document.createElement("p");
// insere no final dos filhos (retorna undefined)
document.body.append(el);
// insere no início dos filhos (retorna undefined)
document.body.prepend(el);
// adiciona ou atualiza um atributo (retorna undefined)
el.setAttribute("id", "x");
```

Selecionando elementos

console.js

```
// ID único → Element | null
document.getElementById("login");
// seletor CSS, o primeiro → Element | null
document.querySelector("h1");
// seletor CSS, todos → NodeList
document.querySelectorAll("li");
// classe CSS, todos → HTMLCollection
document.getElementsByClassName("btn");
// tag HTML, todos → HTMLCollection
document.getElementsByTagName("p");
```

! `HTMLCollection` e `NodeList` são **array-like** — não são arrays puros de JS.

Atributos id e class

id — único no documento
- no máximo um id por elemento
- nenhum outro elemento repete
- usado com `getElementById`

class — reutilizável
- pode se repetir em vários elementos
- um elemento pode ter várias
- base pra aplicar estilo com CSS

Na próxima aula, a diferença entre os dois vira o assunto central.

Servidor, DOM e Vendor API.

Próximas aulas: **CSS e classes** (estilizar via `class`), o **objeto** `window` (aula de navegadores) e **views dinâmicas** (conteúdo gerado pelo backend a cada requisição).

tarefa

TF 01: Servidor de Arquivos Estáticos e DOM

Entrega pelo formulário abaixo —
docs.google.com/forms/d/e/1FAIpQLScaZga2FaqkJQrMD6Ohx1a1Wrz-8cScR8_F0ubu4dE2eaTnPw/viewform